Robô Portal Família

Sistema automatizado de sincronização de dados de pacientes entre Oracle (hospital) e Postgres (Portal da Família).

📋 Explicação para Leigos (Linguagem Simplificada)

O que este programa faz?

Este programa é um "robô" automático que copia informações de pacientes de um banco de dados Oracle (usado pelo hospital) para outro banco de dados Postgres (usado pelo Portal da Família). Ele funciona como uma ponte entre dois sistemas.

Como funciona passo a passo?

1. Inicialização (Preparação)

- O programa começa lendo as configurações de acesso aos dois bancos de dados (Oracle e Postgres) de um arquivo chamado `.env`.
- Ele prepara os registros de log (como um diário) para anotar tudo que acontece.
- Conecta ao Oracle usando um software especial chamado "Instant Client" que permite a comunicação.

2. Busca de Pacientes (Leitura)

- O robô se conecta ao banco Oracle e busca todos os pacientes que estão na lista de "Pacientes com Visitas".
- Ele pega informações como: nome, CPF, telefone, email, data de nascimento, tipo de atendimento, etc.

3. Tratamento dos Dados (Limpeza e Organização)

Para cada paciente encontrado, o robô:

- Limpa os dados (remove espaços extras, organiza informações)
- Cria uma senha automática usando os últimos 6 números do CPF
- Transforma essa senha em um código secreto (hash) impossível de reverter
- Se faltar algum dado (como email), ele inventa um padrão (CPF@naoinformado.com)

4. Verificação de Duplicatas (Evitar Repetição)

- Antes de inserir, o robô verifica no Postgres quais combinações **Atendimento + Tipo/Profissional** já existem.
- Se a mesma combinação já existe, ele ignora; se for um novo tipo ou profissional para o mesmo atendimento, ele insere.

5. Inserção no Postgres (Gravação)

- Para cada paciente novo, o robô insere uma linha completa na tabela do Postgres

- Se der erro em algum paciente específico, ele anota no log de erros e continua com os outros
- Ao final, mostra quantos foram encontrados, quantos foram inseridos e quantos deram erro

6. Repetição Automática (Loop)

- Depois de processar tudo, o robô espera 1 hora
- Depois de 1 hora, ele repete todo o processo novamente
- Isso continua para sempre, até que alguém desligue o programa ou o servidor

Onde ficam os registros?

- **logs/info.log**: Registra resumos de cada execução (quantos pacientes processou)
- **logs/errors.log**: Registra apenas os erros detalhados quando algo dá errado

---

🔧 Explicação Técnica (Linguagem Especializada)

Arquitetura e Fluxo de Execução

1. Configuração e Inicialização do Ambiente

```
BASE_DIR / settings / .env → carregamento de variáveis via dotenv
```

- Carrega parâmetros de conexão Oracle (host, port, service_name, user, password, schema, view)
- Carrega parâmetros de conexão Postgres (host, port, database, schema, table, user, password)
- Resolve dinamicamente o caminho do Oracle Instant Client baseado no SO (Windows/Linux)
- Configura logging estruturado com handlers múltiplos:
  - StreamHandler (stdout) → INFO level
  - FileHandler (logs/info.log) → INFO level
  - FileHandler (logs/errors.log) → ERROR level

2. Inicialização do Driver Oracle (Thick Mode)

```
oracledb.init_oracle_client(lib_dir=INSTANT_CLIENT_PATH)
```

- Modo thick obrigatório para Instant Client
- Fallback chain: `ORACLE_CLIENT_WINDOWS/LINUX` → `ORACLE_INSTANT_CLIENT` → defaults
- Validação de existência do diretório e dependências (oci.dll no Windows, libaio.so no Linux)

3. ETL Pipeline - Extract

```
fetch_oracle_rows() → Generator[Dict]
```

- Conexão via `oracledb.connect()` com parâmetros explícitos (host/port/service_name)

- Query: `SELECT * FROM {ORACLE_SCHEMA}.{ORACLE_VIEW}`
- Streaming via `cursor.fetchmany(500)` para otimização de memória
- Retorna generator de dicts com chaves em uppercase (normalização Oracle)

4. ETL Pipeline - Transform

```
transformar_registro(raw: Dict) → Dict
```

Operações de transformação:

- String normalization: `strip()` em todos os campos texto
- Field mapping com fallback chain (ex: CPF → NRO_CPF → NR_CPF)
- Derivação de `PatientType` **composto**: tipo base (prefixo de TIPOVISITA) + especialidade/profissional (ex.: `ID - Medico`)
- Email fallback: `{CPF}@naoinformado.com` se ausente
- Geração UUID v4 para `IdCadRegistration`
- Timestamp `RegistrationDate` com `datetime.now()`

Argon2id Hash Generation:

```
gerar_senha_e_hash(cpf: str) → Tuple[str, str]
```

- Algoritmo: Argon2id (Type.ID)
- Parâmetros: time_cost=4, memory_cost=65536 (64MB), parallelism=8
- Senha derivada: últimos 6 dígitos do CPF após regex `r"[^0-9]"`
- Salt: 16 bytes aleatórios via `os.urandom()`
- Output: (hash_base64, salt_base64)

5. Deduplicação por Chave Composta

```
buscar_chaves_existentes_postgres() → Set[(IdAdimission, PatientType)]
```

- Query: `SELECT "IdAdimission", "PatientType" FROM {table}`
- Filtro em memória: ignora se `(IdAdimission, PatientType)` já existe no Postgres ou já apareceu no mesmo batch (view retornando duplicadas)
- Objetivo: permitir múltiplos registros para o mesmo atendimento, desde que com tipo/profissional distinto

6. ETL Pipeline - Load

```
inserir_postgres(records: List[Dict]) → Tuple[int, int]
```

- Conexão: `psycopg2.connect()` com autocommit=True (inserções independentes)
- SQL: `INSERT INTO {schema}.{table} ({cols}) VALUES ({placeholders})`
- Quoted identifiers para case-sensitive columns (preserva PascalCase)
- Error handling individual: try/except por registro, log em ERROR level
- Retorno: tuple (sucessos, falhas)

7. Orquestração e Scheduling

```
run_pipeline(watch_interval: int)
```

- Loop infinito: `while True`
- Interval: 3600s (1 hora)
- Execução sequencial por ciclo:
  1. `processar_batch()` → lê Oracle, filtra por chave composta e insere
  2. Logging de resumo
  3. `time.sleep(watch_interval)`

8. Tratamento de Erros e Logging

- **Transformação**: `logger.error()` para falhas em `transformar_registro()`, continua processamento
- **Inserção**: `logger.error()` para falhas em `cur.execute()`, incrementa contador de falhas
- **Níveis de log**:
  - INFO: contadores agregados (encontrados/inseridos/falhas)
  - ERROR: detalhes de exceção + IdAdimission afetado

9. Deployment e Persistência

- Modo standalone: `python robo_portal_familia.py` (processo bloqueante)
- Containerizado: Dockerfile + docker-compose.yml com `restart: unless-stopped`
- State management: stateless entre ciclos (dedupe via query ao Postgres)
- Log rotation: não implementada (requer configuração externa via logrotate/Docker volumes)

Características Técnicas

- **Concorrência**: Single-threaded, blocking I/O
- **Idempotência**: Parcial (dedupe window evita duplicatas imediatas)
- **Transações**: Individual por INSERT (autocommit), não usa batch transactions
- **Escalabilidade**: Limitada pelo single-process, adequado para volumes médios (<10k registros/hora)
- **Tolerância a falhas**: Graceful degradation (falhas individuais não interrompem batch)

🚀 Como Usar

Requisitos

- Python 3.11+
- Oracle Instant Client 23.x

- Acesso aos bancos Oracle e Postgres

Instalação

1. Clone o repositório
2. Crie ambiente virtual:

```
python -m venv .venv
.venv\Scripts\activate  # Windows
source .venv/bin/activate  # Linux
```

3. Instale dependências:

```
pip install -r requirements.txt
```

4. Configure `settings/.env` com as credenciais dos bancos

5. Configure o caminho do Oracle Instant Client no `.env`:

```
ORACLE_CLIENT_WINDOWS=C:\path\to\instantclient_23_0
# ou
ORACLE_CLIENT_LINUX=/opt/oracle/instantclient_23_0
```

Execução

Modo Standalone

```
python robo_portal_familia.py
```

Modo Docker

```
docker compose up -d
docker compose logs -f robo
```

📊 Logs

- **logs/info.log**: Resumo de execuções (contadores)
- **logs/errors.log**: Detalhamento de falhas

Exemplo de log INFO:

```
2026-01-02 14:30:00 [INFO] --- Ciclo 1 iniciado ---
2026-01-02 14:31:15 [INFO] Registros encontrados: 150 | Inseridos: 145 | Falhas: 0
2026-01-02 14:31:15 [INFO] Próximo ciclo em 3600 segundos
```

Exemplo de log ERROR:

```
2026-01-02 14:31:10 [ERROR] Falha ao inserir registro - IdAdimission=12345 | Erro:
duplicate key value
```

Estrutura do Projeto

```
Robo - Portal Familia/
├── robo_portal_familia.py   # Script principal
├── requirements.txt          # Dependências Python
├── settings/
│   └── .env                  # Configurações (não comitar)
├── logs/
│   ├── info.log             # Log de execuções
│   └── errors.log           # Log de erros
├── Dockerfile               # Imagem Docker
└── docker-compose.yml       # Orquestração Docker
```

Segurança

- Senhas hashadas com Argon2id (algoritmo resistente a GPU/ASIC attacks)
- Credenciais armazenadas em `.env` (excluído do Git via `.gitignore`)
- Conexões diretas sem armazenamento intermediário de senhas em texto plano

---

Licença

Projeto interno - DBC Company

Resumo Executivo: Sistema ETL autônomo com polling horário, normalização de dados, hashing criptográfico de senhas e deduplicação por chave composta (Atendimento + Tipo/Profissional), operando como daemon para sincronização unidirecional Oracle→Postgres.